

Matías Eduardo Montané Gana

Σ Nota	5.812056434688
# Failed E1-BasicCombat	0
# Failed E1-InvalidTeams	0
# Failed E1-RandomBasicCombat	0
Σ Puntaje Clean Code	5.63
# Cap. 2	1.93
# Cap. 3	2.2
Σ Puntaje Tests	6

Comentarios Generales

- En algunos métodos retornas `null` y eso te hace tener que verificar el retorno, esto puede evitarlo haciendo que tiren una `Exception`. Aunque no siempre es necesario lanzar una exception.
- En ningún momento vi que agregases las `PassiveSkills` a tus unidades.

Cap. 2: Meaningful Names [1.93 / 2.0]

Descuentos

Nombre Descriptivo [-0.00]

- Se hace buen uso de nombres ✓

Nombre de Método [-0.00]

- No se tienen métodos cuyo nombre no incluya un verbo ✓

Nombre de Clase, Variable y Atributos [-0.01]

- [-0.01] `CombatManager.cs:battleEnded`
 - Debería ser `hasBattleEnded` ya que es una variable booleana.

No encapsular constantes [-0.04]

- [-0.01] `TeamLoader.cs:AreTeamSizesValid`
 - Comparas con 1, 4 y 5, siendo estas constantes las cantidades máximas y mínimas de cada equipo, deberías encapsularlas como por ejemplo `MaxTeamSize`.
- [-0.01] `TeamLoader.cs:TryLoadActiveSkills`
 - Usas un 8 como el tamaño máximo de habilidades que puedes tener, es decir el `MaxActiveSkills` o `MaxAllowedActiveSkill`.
- [-0.01] `CombatManager.cs:HandleTravelerTurn`
 - Usas "1", "2", "3", "4" para llamar a tus acciones posibles, deberías usar un enum, porque en futuras entregas no solamente recibirás output desde la terminal, además es más entendible `input = Action.RunAway=` que `input = "4"=`.
- [-0.01] `CombatManager.cs:CalculatePhysicalDamage`
 - El modificador de ataque básico no está encapsulado; solo está 1.3.

Una palabra por concepto [-0.02]

- [-0.02] `CombatCenter.cs`
 - `Handle` y `Process` hacen referencia al mismo concepto de manejar una situación, mientras que `Execute` y `Resolve` se refieren a ejecutar, debes quedarte con solo uno de cada, por ejemplo `Handle` y `Execute`.

Cap. 3: Functions [2.20 / 2.5]

- 🎁 **Condición punto base:** No se tiene ningún método de más de 60 líneas. Cumple condición, tendrás como mínimo 1 punto en el capítulo ✅

Descuentos

- Las siguientes clases , violan el CQS al retornar y realizar una acción.

Hacer solo una cosa [-0.17]

- [-0.1] `TeamLoader.cs:TryAddTravelerToTeam` , `TryAddBeastToTeam` , `TryLoadActiveSkills`
 - Violan el CQS, y por ende hacen más de una acción. Además de intentar agregar o cargar, retornan si esto fue posible o no.
 - También `ProcessSingleTurn` , `HandleTravelerTurn` , `ExecuteBasicAttack` , `PromptWeaponSelection` , sufren de esto, e incluve otros métodos más.
- [-0.03] `CombatManager.cs:StartCombat`
 - No solamente lo inicia, sino que hace todo el combate, manejando también los "turnos":

```

        while (turnQueue.Count > 0)
        {
            if (CheckWinCondition(out string innerWin
innerMessage))
            {
                PrintMessageWithSeparator(innerWinne
erMessage);
                return;
            }

            bool battleEnded = ProcessSingleTurn(tu
rnQueue);
            if (battleEnded) return;
        }

```

Esto se podría encapsular en su propio método, por ejemplo `HandleTurns` .

- [-0.02] `CombatManager.cs:HandleTravelerTurn`
 - Maneja el input y output, además de manejar explícitamente el caso de la huida, se puede separar en 2 métodos, `view.AskTravelerAnOption` que solo nos entrega la opción y luego `ActionHandler.GetActionFrom(option)` , `ActionHandler` sería una factory. Y así trabajar la huida como una `Action` .

- [-0.01] `CombatManager.cs:ProcessSingleTurn`
 - El manejo de la cola debería ser parte de quien llama a `processSingleTurn`.
- [-0.01] `Game.cs:LoadGameDataAndTeams`
 - El nombre indica la falta, y la falta de que cargues todo a variables te obliga a pasarlo todo dentro de parametros, esta carga puede ser hecha en un clase especifica, de esta forma eliminas esos 5 parametros por 1. Y llevas la carga de datos a otra clase.

Código Duplicado [-0.02]

- [-0.01] `TeamLoader.cs:TryLoadPassiveSkills` y `TryLoadActiveSkills`
 - Las únicas difencias son los separados y que `ActiveSkills` las carga de verdad. Deberías tener una función general que dado los separadores te entregue netamente las skills y tu por fuera las cargues. Así evitando repetir código.
- [-0.01] `CombatManager.cs:StartCombat`
 - Se usa el siguiente código que podría evitarse siendo una función o mejor aún replanteandonos cuándo se debe imprimir dicho dialogo, y netamente hacer el check y detener la ejecución de los loops.

```
if (CheckWinCondition(out string winnerMessage))
{
    PrintMessageWithSeparator(winnerMessage);
    return;
}
```

Argumentos [-0.06]

- [-0.03] `TeamLoader.cs:LoadTeam`
 - Recibe 5 argumentos, y algunos de estos podrían volverse atributos y parsarselos al constructor, como por ejemplo:

`characters, enemies, activeSkills, passiveSkills`, renombrando el método a `LoadTeamFrom(selectedFile)`.

- [-0.00] `TeamLoader.cs:ParseTeamsFromLines`
 - Recibe 5 argumentos, esto se puede evitar con la misma idea anterior.
- [-0.00] `TeamLoader.cs:ParseTraveler`
 - Recibe 4 argumentos.
- [-0.03] `CombatManager.cs:CheckWinCondition`
 - Usa un output argument.

Condiciones en if y while [-0.04]

- [-0.02] `CombatManager.cs:PromptWeaponSelection`
 - La condición `int.TryParse(_view.ReadLine(), out int choice) && choice > 0 && choice <= traveler.Weapons.Count` debería ser `_isWeaponChoiceValid`.
- [-0.02] `Combatmanager.cs:PromptTargetSelection`
 - `int.TryParse(_view.ReadLine(), out int choice) && choice > 0 && choice <= aliveEnemies.Count` debería ser `_isTargetValid`.

Funciones booleanas [0.00]

- No se encuentran funciones que reciban bool 

Indentación [-0.01]

- [-0.01] `CombatManager.cs:StartCombat`
 - Tiene 3 niveles de indentación, siendo el último innecesario, porque se puede encapsular en `handleTurns`.